PLATFORM FOR AUTONOMOUS INDOOR NAVIGATION USING SLAM, META-RL, AND MULTI-ROBOT COLLABORATION

25-26J-497

Project Proposal Report

Thilochana J M

BSc (Hons) Degree in Information Technology

Specializing in Software Engineering

Department of Software Engineering

Sri Lankan Institute of Information Technology Sri Lanka

August 2025

PLATFORM FOR AUTONOMOUS INDOOR NAVIGATION USING SLAM, META-RL, AND MULTI-ROBOT COLLABORATION

25-26J-497

Project Proposal Report

Thilochana J M – IT22899224

Supervisor: Mr. Samadhi Rathnayake

Co-Supervisor: Mr. Amila Alexander

BSc (Hons) Degree in Information Technology

Specializing in Software Engineering

Department of Software Engineering

August 2025

DECLARATION

I declare that this is my own work, and this proposal does not incorporate without acknowledgement any material previously submitted for a degree or diploma in any other university or Institute of higher learning and to the best of our knowledge and belief it does not contain any material previously published or written by another person except where the acknowledgement is made in the text.

Name	Student ID	Signature
Thilochana J M	IT22899224	

The above candidates are carrying out research for the undergraduate Dissertation under my supervision.

28/8/25

---------- ----------

Signature of the Supervisor Date

---------- ----------

Signature of the Co-Supervisor Date

ABSTRACT

Autonomous navigation in dynamic and unpredictable environments requires systems that can adapt their decision-making in real time while maintaining accurate environmental awareness. The approach outlined in this work combines Simultaneous Localization and Mapping with Meta-Reinforcement Learning (Meta-RL) in a decentralized yet cloud-assisted architecture, enabling multi-robot teams to operate efficiently in changing conditions. The design focuses on continuous policy adaptation, distributed data processing, and human-in-the-loop control to enhance scalability and operational resilience. At the edge layer, autonomous robots equipped with LiDAR and camera systems gather detailed environmental data while navigating. Each robot processes sensor inputs locally for immediate obstacle avoidance and short-term path planning. Data is then transmitted to a local edge server, which merges information from multiple robots, filters noise, and compiles preliminary maps. This step reduces the computational and bandwidth burden on the robots themselves, ensuring low-latency responses in time-sensitive scenarios. The cloud processing layer manages global computation tasks, including high-fidelity SLAM processing, map fusion across all deployed robots, and centralized policy training through Meta-RL. By integrating multi-robot data into a unified 3D map, the system gains a shared situational understanding, which the Meta-RL module uses to refine navigation strategies. The cloud environment continuously trains and updates navigation models, pushing optimized policies back to the edge devices without requiring manual reprogramming. This loop allows the robots to learn from both past experiences and newly encountered environments. A dedicated web-based visualization interface gives operators real-time access to robot locations, sensor data, and environmental maps. This interface supports live monitoring as well as human feedback input, which is incorporated into future policy adjustments. This design ensures that domain expertise can be combined with automated learning, leading to more robust and context-aware navigation.

Keywords - Autonomous Navigation, Multi-Robot Systems, SLAM, Meta-Reinforcement, Edge-Cloud Architecture

TABLE OF CONTENT

Declaration ..... **Error! Bookmark not defined.**

Abstract i

List of tables v

List of figures vi

List of abbreviations vii

1 Introduction 1

1.1 Background and Literature Survey 2

1.2 Research Gap 6

1.3 Research Problem 9

2 Research Objectives 12

2.1 Main Objectives 12

2.2 Specific Objectives 13

2.2.1 Development of a Model Training Platform 13

2.2.2 Development of the Deployment Platform 13

2.2.3 Development of the Prototype Robot 14

2.2.4 Real-time Monitoring and Performance Evaluation 14

3 Methodology 15

3.1 System Overview 15

3.2 Software Solutions 16

3.2.1 Requirement Gathering and Analysis 16

3.2.2 Feasibility Study 18

Schedule Feasibility 18

Technical Feasibility 18

3.3 Tasks and Sub-Tasks 20

3.4 Testing and Validation 22
3.5 Data Collection 22
3.6 Time Frame & Schedule 23
4 Project Requirements 24
4.1.1 Functional Requirements 24
4.1.2 Non-Functional Requirements 25
4.1.3 User Requirements 26
4.1.4 System Requirements 26
References 27

LIST OF TABLES

Table 1.1 Core Challenges in Multi-Robot System Deployment..... 5
Table 1.2 Research Gap Analysis..... 8

LIST OF FIGURES

Figure 3.1 System Architecture Diagram 15
Figure 3.2 Sub Component Diagram 20
Figure 3.3: Gantt Chart 23

LIST OF ABBREVIATIONS

Abbreviation	Full Form
AMR	Autonomous Mobile Robot
AGV	Automated Guided Vehicle
SLAM	Simultaneous Localization and Mapping
Meta-RL	Meta-Reinforcement Learning
MAML	Model-Agnostic Meta-Learning
LiDAR	Light Detection and Ranging
RGB-D	Red, Green, Blue – Depth
IMU	Inertial Measurement Unit
CORB-SLAM	Collaborative ORB-SLAM
DDF-SLAM	Distributed Data Fusion SLAM
GPU	Graphics Processing Unit
TPU	Tensor Processing Unit
DRL	Deep Reinforcement Learning
3D	Three-Dimensional
UAV	Unmanned Aerial Vehicle
API	Application Programming Interface
IoT	Internet of Things

1 INTRODUCTION

Autonomous indoor navigation in unknown and dynamic environments has long been a challenge in robotics, but it is becoming increasingly important in areas such as smart homes, automated warehouses and disaster recovery scenarios. The ability of robots to travel successfully without prior maps is significantly relied on SLAM, a technique that allows for real time environment reconstruction while tracking the robot's position [1]. SLAM has made significant advances with optical, LiDAR and multi sensor fusion techniques providing robust performance in controlled environments. However most traditional SLAM algorithms assume static or semi static surroundings and a single robot operational model, limiting their effectiveness in real world applications where settings are fundamentally dynamic [4].

Dynamic components including moving individuals, moving furniture, clear surfaces and reflective materials pose significant problems to typical SLAM systems. These issues appear as false loop closures, map inconsistencies, and loss of localization accuracy which could risk navigation safety and efficiency. Rapid adaptation is crucial in highly dynamic environments, yet standard rule-based path planning and control systems frequently fail to respond successfully [5].

Meta-RL has emerged as a potential method for rapid robotic adaptability. Unlike the traditional reinforcement learning methods, Meta-RL allows systems to make assumptions from previous experiences and quickly modify navigation strategies to the new unknown environments [9]. Meta-RL has demonstrated great potential for single robot navigation but how it can be used in a multi-robot collaborative context is mostly unexplored due to various reasons. This gap is significant since collaborative navigation offers various advantages such as simultaneous exploration for efficient mapping, shared situational awareness and resilience through distributed decision-making [6].

The multi-robot collaboration framework is a key component for these advantages. It allows real time map sharing, task allocation, conflict resolution and synchronized exploration. By merging local maps into a consistent global map [8], robots can leverage one another's discoveries. Our framework operationalizes these requirements

with decentralized coordination and robust map fusion and scheduling mechanisms, designed for heterogeneous robot teams [7].

This proposal introduces an integrated platform for Autonomous Indoor Navigation by combining advanced SLAM with Meta-RL based adaptation engine, a decentralized multi robot collaborative framework and a 3D mapping and visualization platform. The system utilizes an edge-cloud architecture, in which lightweight robots with simple sensors transporting heavy computations like SLAM processing, map fusion, and policy optimization to a centralized server. This design offers scalability, cost-efficiency, and interoperability with a variety of robot types while preserving real-time performance [6].

From a software engineering approach, the platform encourages flexibility, reusability, and scalability to facilitate the integration of emerging algorithms, sensors, and robotic platforms. Along with SLAM, Meta-RL, and the multi-robot cooperation architecture, the 3D Mapping & Visualization Platform offers an easy-to-use interface for real-time monitoring and debugging for operators. These components improve autonomous robotics by allowing multi-robot teams to move collaboratively, adapt to dynamic changes, and keep consistent global maps in complicated interior environments. The suggested solution bridges the gap between academic research and industry implementation, increasing logistical efficiency, ensuring safety in hazardous areas, and allowing new robotics applications in human-centric environments.

1.1 Background and Literature Survey

The evolution of autonomous systems has altered the way indoor logistics, inspection and service activities are carried out. Historically, tasks like item transportation and inventory management relied significantly on human labor which was inefficient, physically demanding and error prone. Automated Guided Vehicles (AGVs) were the first wave of automation to enable repeated activities in organized and controlled environments. However, AGVs depended on permanent navigation infrastructure, such as magnetic strips or painted floor lines, rendering them inefficient in the places with layout changes and dynamic obstacles [1].

The introduction of Autonomous Mobile Robots (AMRs) marked a significant leap forward. AMRs can dynamically detect and traverse new surroundings by utilizing advanced sensing methods such as LiDAR, RGB-D cameras and inertial measurement units (IMUs) as well as SLAM algorithms [3]. This enables its usage in complicated, real-world environments such as warehouses, hospitals and smart buildings. However, in large-scale applications a single AMR is insufficient. The true potential lies in deploying fleets of robots that can work together to explore and operate more effectively. This transition from a single agent to multi agent system creates a new set of challenges that define the boundary of current robotics research [7].

Indoor environments are hardly static because moving people, rearranging furniture and temporary obstacles cause a shift in distribution between training and operating settings. While Deep Reinforcement Learning (DRL) has shown ordinary DRL agents require considerable training in new settings, making them not suitable for applications that require immediate attention. Meta-RL solves this problem by allowing agents to "learn how to learn" and change rules with little new knowledge [10]. For example, Google research conducted in 2020 demonstrated fast adaptability in legged robots through evolutionary meta-learning [11]. However, developing whole lifecycle platforms from simulation-based training to real world deployments and constant fine-tuning is still an unexplored topic.

Scaling from a single robot to a collaborative team raises serious distributed system challenges. To outperform a single agent robot, a team of robots must coordinate their activities to achieve a shared goal, such as exploring an unexplored region. This requires addressing several key challenges:

- **Distributed Model Training and Deployment**: In multi-robot systems, efficiently training and deploying machine learning models across the team presents additional challenges. Models may need to be updated based on collective data from all robots, requiring synchronization and bandwidth-efficient distribution. Ensuring that each robot runs the latest optimized model while minimizing communication overhead and latency is crucial for maintaining coordinated behavior in real time.

- **Communication and Data Fusion**: Agents must effectively share sensor data and local maps with each other. Combining these incomplete maps into a single global map is a difficult data fusion task [6]. Systems such as CORB-SLAM have created frameworks for this but they frequently rely on protocols that may struggle with scalability and network instability.
- **Dynamic Task Allocation**: Dynamic task allocation is appointing different agents for different areas of a building to explore in order to increase coverage and reduce redundancy. Cooperative navigation techniques are vital for efficiency but building decentralized algorithms that lead to globally optimum behavior remains a persistent difficulty [8].
- **Conflict Resolution**: Managing path conflicts and right-of-way to prevent collisions and deadlocks in shared environments.

These challenges require robust communication protocols, consensus algorithms, and lightweight yet scalable data-sharing strategies.

A multi-robot fleet generates a continuous stream of high-dimensional real-time data which includes raw sensor feeds, SLAM-generated maps, navigation choices, job allocations and inter-robot communication. Without the appropriate tools, this data can become overwhelming for operators, which can make the system impossible to analyze, diagnose and optimize. Existing interfaces often provide only basic 2D positional data, lacking advanced features such as collision heatmaps, historical performance analytics, and interactive map annotations [12].

The absence of integrated visualization and diagnostic tools in many existing multi-robot frameworks is a major obstacle to scaling. Without them, troubleshooting involves manual log analysis, which is time-consuming and prone to error.

To address this difficulty, a 3D Mapping & Visualization Platform must be developed that not only depicts real-time global maps and robot data, but also includes historical analytics, performance heatmaps, and interactive controls. Such a system converts raw sensor and navigation data into usable insights, allowing operators to make quick decisions, save downtime, and retain confidence in the system's performance even if the number of robots and mission complexity increases.

Table 1.1 Core Challenges in Multi-Robot System Deployment

Factor	Practical Effect on Robots	Typical Symptom	Impact (1-5)
Distributed Model Training and Deployment	Delays or inconsistencies in model updates across robots; inefficient use of cloud resources	Robots act inconsistently and outdated models lead to poor coordination or decision-making	5
Real-Time Adaptation and Dynamic Obstacles	Inability to adjust to new or moving obstacles	Collisions, inefficient pathing, operational freezes	5
Multi-Robot Coordination and Scalability	Robots performing redundant work or interfering with each other	Inefficient area coverage, inconsistent global maps, bottlenecks	5
System Complexity and Operational Oversight	Inability for human operators to monitor, debug, or manage the fleet	Prolonged downtime, difficulty diagnosing failures, lack of trust in the system	4

The available literature typically addresses these issues in isolation. Meta-RL research may produce a highly adaptable agent, but it fails to investigate how it operates in multi-agent systems. Working on collaborative SLAM may help with map merging, but it assumes robots with basic, non-adaptive behaviors. The key gap therefore is the lack of a single, integrated platform to address these concerns comprehensively. Without a strong framework to regulate these interactions, a team of robots may be less efficient than a single agent. This difficulty is essentially a distributed systems engineering problem, requiring solutions that assure data consistency and allow for dynamic resource allocation across independent agents.

1.2 Research Gap

Autonomous navigation in indoor and semi-structured environments is a rapidly evolving field that combines robotic perception, planning, and control. Several research areas contribute significantly, including SLAM, reinforcement learning, multi-robot collaboration, and dynamic change detection. While individual studies demonstrate strong progress, a detailed comparative analysis reveals persistent gaps, particularly when considering real-time adaptability, multi-robot coordination, semantic-aware change detection, and full integration into a unified operational platform.

SLAM is fundamental for autonomous navigation, providing robots with the ability to localize and map unknown environments simultaneously. ORB-SLAM [1] introduced an efficient monocular SLAM system with high-accuracy localization and loop closure capabilities. While ORB-SLAM performs well in static environments, it struggles in dynamic scenarios with moving obstacles, limiting its applicability for real-world operations. RTAB-Map [2] extended SLAM to handle real-time RGB-D data and appearance-based loop closure. However, it is optimized for single-robot operations, and centralized processing limits scalability in multi-robot deployments. Cartographer [3] offers real-time 2D/3D mapping using LiDAR but has minimal dynamic obstacle handling and high computational requirements for multi-sensor fusion.

While these systems advance mapping accuracy, they do not integrate real-time adaptive decision-making, semantic awareness, or multi-robot collaboration, which are crucial for long-term autonomous operations in evolving environments. Reinforcement learning enables robots to learn optimal navigation strategies. Meta-reinforcement learning, particularly MAML [4], allows robots to rapidly adapt policies to unseen environments. Meta-RL for obstacle avoidance in simulated scenarios [5], showing improvements over standard RL. Despite this, most research remains simulation-based, lacking integration with real-world SLAM, real-time sensor feedback, and multi-robot coordination. Existing RL approaches also fail to handle sensor noise, communication delays, and the constraints of real-time control. Consequently, there is a critical gap in developing Meta-RL strategies that operate

seamlessly within a real-world SLAM-based, multi-robot system, enabling dynamic obstacle-aware navigation. Ulti-robot systems can significantly improve exploration efficiency and coverage. Distributed SLAM frameworks like CORB-SLAM [6] and DDF-SLAM [7] support map fusion and collaborative data sharing among multiple robots. These methods enhance global map consistency but rely heavily on network reliability, assume homogeneous robot teams, and often use centralized control. Task allocation and conflict resolution are typically rigid or static. Furthermore, most frameworks lack integration with adaptive learning policies or dynamic obstacle-aware navigation**,** reducing their utility in dynamic real-world scenarios.

Dynamic environments necessitate robust change detection to maintain accurate maps. Focus on methods for detecting structural [9] changes using multi-sensor [8] fusion and semantic labeling. While effective, these approaches are mostly offline or single-robot [13] focused and computationally intensive, limiting real-time deployment. There is no scalable system that supports real-time, decentralized semantic-aware map updates, particularly for multi-robot teams that need consistent and coordinated navigation in evolving indoor environments. Existing research demonstrates significant progress in isolated domains but fails to provide a unified solution. SLAM, Meta-RL, multi-robot collaboration, and dynamic change detection largely operate in isolation. No study fully integrates these capabilities into a comprehensive autonomous navigation platform that supports:

- Real-time adaptive navigation via Meta-RL,
- Decentralized multi-robot collaboration with dynamic task allocation and conflict resolution,
- Continuous 3D SLAM mapping with semantic awareness, and
- Real-time, version-controlled change detection integrated into operational workflows.

Bridging these gaps is critical for practical applications such as warehouse automation, indoor monitoring, and disaster response, where robots must adaptively navigate, collaborate, and update shared maps in real-time.

Table 1.2 Research Gap Analysis

Research Study	Focus Area	Strengths	Limitations / Gap	Relation to Proposed Comprehensive Platform	Reference
ORB-SLAM	3D SLAM	Accurate localization, loop closure	Static environments single robot lacks dynamic adaptation	Forms SLAM backbone; extended for multi-robot and dynamic updates	[1]
RTAB-Map	Real-time RGB-D SLAM	Loop closure; real-time mapping	Limited scalability; single-robot focus; centralized	Used as backend for collaborative multi-robot SLAM	[2]
Cartographer	LiDAR SLAM	Real-time 2D/3D mapping	High computation and minimal dynamic obstacle handling	Integrated into hybrid platform with adaptive obstacle awareness	[3]
MAML	Meta-RL	Rapid generalization to unseen tasks	Simulation only; lacks real-world integration; single robot	Provides Meta-RL adaptive navigation engine	[4]
Obstacle Avoidance	Meta-RL obstacle avoidance	Adaptive policies in simulation	Single robot; no multi-robot coordination and ignores sensor noise	Basis for real-time Meta-RL in multi-robot setup	[5]
CORB-SLAM	Multi-robot distributed SLAM	Map fusion; distributed architecture	Network-dependent; centralized task allocation; no adaptive policies	Inspiration for decentralized multi-robot collaboration framework	[6]
DDF-SLAM	Multi-robot SLAM	Data sharing; distributed mapping	Homogeneous robots; limited dynamic adaptation	Adapted for heterogeneous robot teams with adaptive path planning	[7]

From the comparative analysis, the key gaps are:

1. Real-Time Adaptive Navigation Gap**:** Most Meta-RL and RL approaches are simulation-based and lack real-world SLAM integration, limiting adaptive obstacle-aware navigation [5].
2. Multi-Robot Coordination Gap**:** Existing multi-robot SLAM systems depend on centralized control, homogeneous robots, and static task allocation, without integration of adaptive learning [7].
3. Dynamic Change Detection Gap**:** Semantic-aware change detection approaches are offline, single-robot focused, and computationally expensive [8].
4. Integration Gap: No research integrates SLAM, Meta-RL, multi-robot collaboration, and semantic-aware change detection into a comprehensive autonomous navigation platform**.**

The gaps clearly justify the need for a holistic platform that unifies these capabilities. The proposed system addresses these gaps by providing real-time adaptive navigation, decentralized multi-robot collaboration, continuous 3D SLAM mapping, and semantic-aware change detection. This comprehensive platform ensures robustness, scalability, and operational efficiency in unknown or evolving indoor environments, overcoming the limitations of existing research.

1.3 Research Problem

The rapid advancement of autonomous mobile robots (AMRs) has significantly improved indoor navigation capabilities, yet numerous challenges remain that hinder their deployment in dynamic and unstructured environments. Traditional Simultaneous Localization and Mapping (SLAM) techniques, such as ORB-SLAM [1] and RTAB-Map [2], have demonstrated high-accuracy mapping and localization in static or structured settings; however, their performance significantly declines when faced with dynamic obstacles, moving furniture, or temporary environmental changes.

Similarly, LiDAR-based mapping systems like Cartographer [3] provide real-time 2D and 3D mapping but are computationally intensive and lack the ability to adapt effectively to real-time dynamic conditions or integrate semantic awareness. While these SLAM systems form the backbone of robotic perception, they typically operate in isolation and do not natively support decentralized multi-robot collaboration or adaptive navigation strategies. In parallel, reinforcement learning approaches, particularly Meta-Reinforcement Learning (Meta-RL) techniques like MAML [4], have enabled rapid adaptation to unseen scenarios, allowing robots to generalize navigation policies to new environments.

Studies such as Zeng et al. [5] demonstrate the potential of Meta-RL for obstacle avoidance in simulated environments. However, these methods are primarily simulation-based, often ignoring real-world constraints like sensor noise, communication delays, and the latency inherent in real-time control. Furthermore, existing Meta-RL implementations rarely consider multi-robot environments where coordination, task allocation, and map-sharing are critical. Multi-robot collaboration frameworks, including CORB-SLAM [6] and DDF-SLAM [7], offer distributed SLAM solutions with map fusion and shared environmental awareness, enhancing global consistency and exploration speed. Nonetheless, these systems rely on centralized task allocation, assume homogeneous robot teams, and lack integration with adaptive Meta-RL policies for real-time obstacle-aware navigation, limiting their scalability and robustness in heterogeneous or large-scale deployments. Another persistent challenge is dynamic change detection. Furthermore, multi-sensor fusion [8]and semantic information to detect environmental changes [9] over time, improving map accuracy and navigation safety. Yet, most existing methods are offline or designed for single-robot scenarios, failing to support real-time, decentralized semantic-aware updates in multi-robot teams.

As a result, robots often operate with outdated maps, leading to inefficiencies and potential collisions. Collectively, these limitations highlight the lack of a unified solution that seamlessly integrates SLAM, Meta-RL adaptive navigation, multi-robot collaboration, and dynamic change [14] detection into a comprehensive autonomous

navigation platform. Such a platform is essential for enabling AMRs to navigate unknown indoor environments efficiently, adapt to dynamic obstacles in real time [15], share mapping and exploration data among heterogeneous teams, and maintain continuously updated semantic-aware maps. Addressing this problem requires a holistic approach that combines scalable software architectures, hybrid edge-cloud processing, robust communication protocols, and modular algorithmic pipelines. By integrating these capabilities, the proposed platform aims to provide an end-to-end solution that ensures real-time adaptability, operational robustness, and scalability, bridging the gap between isolated research contributions and practical deployment in real-world dynamic environments. This research problem directly motivates the design and implementation of a comprehensive autonomous navigation system capable of synergizing 3D SLAM mapping, Meta-RL-based obstacle-aware adaptation, decentralized multi-robot coordination, and semantic-aware change detection, thereby addressing the critical limitations present in the current state of the art.

2 RESEARCH OBJECTIVES

2.1 Main Objectives

The primary aim of this research is to design, develop, and validate a comprehensive autonomous navigation platform capable of operating efficiently in dynamic and unknown indoor environments. The platform will seamlessly integrate adaptive navigation algorithms, robust mapping systems, and multi-robot collaboration strategies into a unified ecosystem. This will involve the combination of advanced SLAM-based mapping, adaptive obstacle avoidance using reinforcement learning techniques, and a decentralized coordination mechanism to ensure scalability and robustness in multi-agent deployments. Furthermore, the platform will incorporate semantic change detection to maintain an up-to-date understanding of the environment, enabling the robots to make informed navigation and task allocation decisions in real time. The system will not only exist in simulation but will also be implemented and tested on a functional prototype robot, bridging the gap between theory and practical application. In addition to core navigation functionalities, the research aims to establish a strong model training pipeline and deployment strategy that allows rapid iteration and improvement of algorithms while ensuring they meet real-world operational constraints.Real-time monitoring tools will be integrated into the system to continuously assess model performance, detect anomalies, and trigger adaptive retraining when necessary. By unifying training, deployment, monitoring, and physical testing into a single framework, the research intends to provide a scalable, reliable, and adaptable solution that can serve as a benchmark for future developments in autonomous navigation for both industrial and service robotics contexts.

2.2 Specific Objectives

2.2.1 Development of a Model Training Platform

The first objective is to build a comprehensive training platform capable of supporting the iterative development of navigation models. This platform will simulate varied indoor environments with dynamic obstacles, allowing for extensive testing of SLAM, path planning, and obstacle avoidance algorithms. It will support reinforcement learning and other adaptive approaches by providing a controlled yet highly variable training environment. The platform will incorporate automated preprocessing pipelines, dataset generation tools, and performance benchmarking modules. By enabling hyperparameter tuning, distributed training, and modular integration of different algorithmic approaches, the platform will foster rapid experimentation and refinement. Its output will be standardized, ensuring that trained models can seamlessly move into the deployment phase without requiring extensive reconfiguration. The training process will be designed to capture edge cases and environmental anomalies, ensuring that models generalize well to real-world operational conditions.

2.2.2 Development of the Deployment Platform

The second objective focuses on establishing a reliable deployment platform that bridges the gap between trained models and operational use. This will involve designing software infrastructure to manage the transfer of models from the training environment to the field without compatibility issues. The platform will handle version control, rollback mechanisms, and over-the-air updates to ensure rapid and safe deployment. It will also provide integration interfaces for hardware, enabling compatibility with a wide range of sensor configurations and computational units. The deployment pipeline will prioritize operational readiness, meaning deployed models must meet predefined benchmarks before activation. This approach ensures minimal downtime and reduces the risks associated with deploying untested updates. The deployment platform will also be designed for scalability, allowing multiple robots or units to be updated and managed simultaneously.

2.2.3 Development of the Prototype Robot

The third objective is to design and construct a functional prototype robot tailored for testing the navigation platform in real-world scenarios. The robot will be equipped with essential sensors such as LiDAR, depth cameras, IMUs, and ultrasonic modules to support mapping and obstacle avoidance. The computational unit will be optimized for running navigation algorithms in real time, balancing processing power with energy efficiency. The robot's chassis and mobility system will be modular to facilitate rapid prototyping and hardware upgrades. Careful consideration will be given to durability and operational flexibility so that the robot can operate in varied indoor settings. The prototype will serve as the main physical testbed for validating navigation models, deployment mechanisms, and monitoring systems, providing crucial feedback that informs iterative improvements in both hardware and software.

2.2.4 Real-time Monitoring and Performance Evaluation

The fourth objective is to create a robust monitoring and evaluation framework that tracks the operational performance of deployed models in real time. This will involve continuous collection of sensor data, navigation metrics, and system health parameters, which will be visualized through an operator dashboard. The system will feature automated anomaly detection to identify deviations from expected performance, triggering alerts or adaptive responses. Performance trends will be logged over time to enable predictive maintenance and gradual model refinement. This monitoring framework will act as a feedback loop to the training platform, ensuring that the system evolves based on real-world operational data. By enabling proactive issue resolution and continuous improvement, the monitoring system will ensure that the autonomous navigation platform remains reliable, efficient, and adaptable to changing environments.

3 METHODOLOGY

Implementing an autonomous navigation platform that includes SLAM, Meta-RL, and multi-robot coordination. The system facilitates decentralized collaboration among robots, allowing for real-time mapping data exchange, synchronized navigation decisions, and conflict resolution in dynamic indoor situations. The workflow follows a staged approach that includes requirement analysis, system design, implementation, testing, and optimization. Each phase addresses issues such as dynamic environmental changes, network dependability, computational restrictions and scalability, ensuring that the framework is adaptable and efficient in real-world applications.

3.1 System Overview

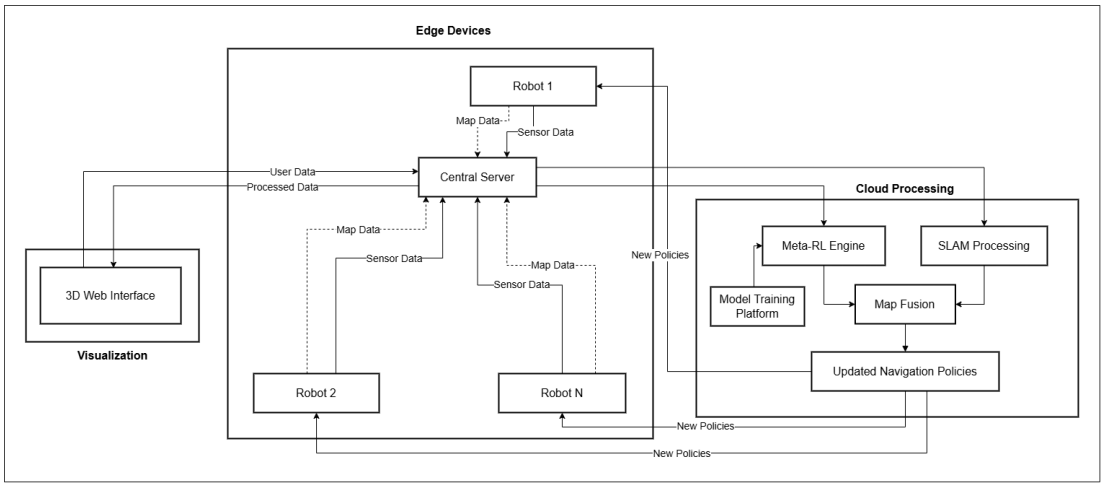

Figure 3.1 System Architecture Diagram

Edge Devices: The framework consists of several robots, each equipped with cameras, LiDAR, and other sensors (e.g. IMUs) for collecting raw maps and sensor data. These edge devices have minimal onboard processing capabilities and are intended primarily to capture and transmit data to the central server rather than perform complex calculations. To ensure high-quality input, robots use robust data gathering techniques, while simple connectivity protocols allow real-time data streaming to the server.

Central Server: The central server functions as the main processing unit, receiving raw map and sensor data from all robots. It performs heavy lifting tasks such as real-

time map creation and fusion, as well as dynamic task allocation using collaborative SLAM techniques. The server collects and interprets incoming data to generate a consistent global map, resolves navigation conflicts, and assigns exploration zones based on robot telemetry (for example, battery condition). User data from the visualization interface is also analyzed here to aid with manual interventions such as work reassignment.

Cloud Processing: The cloud layer provides the system with powerful computing resources, including the Meta-RL Engine, SLAM Processing, Map Fusion, and Modeling Platform modules. The Meta-RL Engine trains adaptive navigation policies and aggregated data from the central server. SLAM Processing uses large-scale optimization approaches to update global maps whereas the Map Fusion module combines local data into a consistent representation that accounts for dynamic changes. The Modeling Platform simulates scenarios to develop new rules, which are then sent to the robots as updated navigation policies using the central server.

Visualization: The 3D Web Interface is a real-time visualization and control platform that receives processed data from a central server. It displays robot movements, global map updates, and job assignments, and includes features such as collision heatmaps. Operators can leverage user data such as priority zone marks to affect job assignments, allowing for greater flexibility in mission management.

This architecture ensures effective data handling and interoperability, with the central server and cloud processing reacting to network fluctuations and changes in the environment. It uses robust SLAM approaches and cooperative navigation strategies, making it appropriate for applications such as warehouse automation and smart houses.

3.2 Software Solutions

3.2.1 Requirement Gathering and Analysis

The development process begins with gathering precise requirements for the model training platform, deployment platform, and prototype robot, ensuring that trained navigation models can transition seamlessly from simulated environments to real-

world execution. Key requirements include determining the computational resources necessary for real-time processing, defining the complexity of simulated environments, and designing a robust communication pipeline between the training system, deployment hardware, and onboard sensors.

During this stage, available LiDAR datasets such as open-source urban mapping and indoor scan repositories are evaluated to quickly test algorithms and assess processing pipelines without immediate reliance on physical hardware. This early experimentation helps identify optimal preprocessing techniques and filtering methods to handle issues such as sensor noise, occlusions, and varying lighting conditions for camera data.

To support informed sensor and algorithm selection, a prototype 3D visualization tool will be developed. This tool will render LiDAR point clouds and camera frames together, enabling real-time inspection of spatial accuracy, alignment quality, and environmental coverage. Such visualization aids in determining the most suitable LiDAR resolution, field of view, and camera lens configuration for the final robot.

Requirement gathering also draws on industry standards, academic literature, and small-scale proof-of-concept trials, ensuring that each component aligns with proven best practices. Potential failure modes such as inaccurate SLAM mapping due to reflective surfaces or delays in model updates due to network latency are identified early to inform mitigation strategies. By finalizing these requirements upfront, the subsequent design and development stages can progress with reduced risk, streamlined integration, and greater confidence in achieving the project's navigation goals.

3.2.2 Feasibility Study

Schedule Feasibility

The proposed project schedule is designed to be both realistic and efficient, following a phased and partially overlapping approach to reduce overall development time. Phase one will focus on the design and implementation of the simulation and model training platform, where synthetic datasets will be generated using simulated LiDAR and camera feeds within Gazebo. This will allow early-stage algorithm development without waiting for hardware readiness. Phase two will begin while phase one is still ongoing, dedicated to the creation of the deployment platform and the initial assembly of the prototype robot. This overlap ensures that software readiness coincides with hardware availability. The third phase will integrate all components and model training outputs, deployment infrastructure, and robot hardware followed by iterative testing in real-world environments. These tests will include varied indoor and semi-outdoor conditions to evaluate model adaptability. By planning for parallel task execution, leveraging existing open-source tools, and setting milestone-based progress reviews, the project maintains a balance between speed and quality assurance. Contingency periods are built into the timeline to address unforeseen hardware delays, software debugging, or integration complexities. The schedule is further strengthened by the use of containerized deployment pipelines, which significantly reduce integration time between the training environment and operational platform.

Technical Feasibility

The technical feasibility of the project is high, as it leverages well-established robotics frameworks and hardware platforms. The prototype robot will be based on an Arduino microcontroller for sensor integration and control, providing a lightweight and cost-effective hardware solution. The model training and inference will be hosted on a cloud platform, enabling powerful computation without relying on onboard GPUs or TPUs. Sensor data, including inputs from a 360° LiDAR sensor and a monocular or stereo camera, will be transmitted from the Arduino to the cloud over a reliable network connection. There, deep learning models will process the data to perform tasks such as

SLAM, object detection, and navigation planning. The cloud-based approach allows the use of computationally demanding algorithms without being constrained by the robot's onboard hardware. The modular design of the system ensures that individual components, such as sensors or communication modules, can be swapped or upgraded without major architectural changes. Standardized sensor message formats and established robotics protocols ensure smooth integration between the Arduino and the cloud services. Additionally, leveraging off-the-shelf sensors and Arduino hardware reduces procurement risks, while the use of open-source cloud tools minimizes software licensing costs.

As a potential upgrade, the Arduino-based robot can be enhanced by integrating an NVIDIA Jetson Orin Nano or similar embedded AI board. This would enable local processing of LiDAR point clouds and camera imagery, reducing reliance on cloud computation and network latency. With sufficient GPU compute power, the onboard board could execute SLAM, object detection, and navigation algorithms in real time, opening the door for more autonomous operation and faster response times. This step would improve system performance while retaining the modular and upgradeable architecture.

Economic Feasibility

Estimating costs for hardware procurement (robot units, sensors, server hardware), cloud computing resources, and software licensing if applicable.

Operational Feasibility

Assessing integration complexity with existing SLAM and Meta-RL systems, operator training requirements, and maintenance demands.

3.3 Tasks and Sub-Tasks

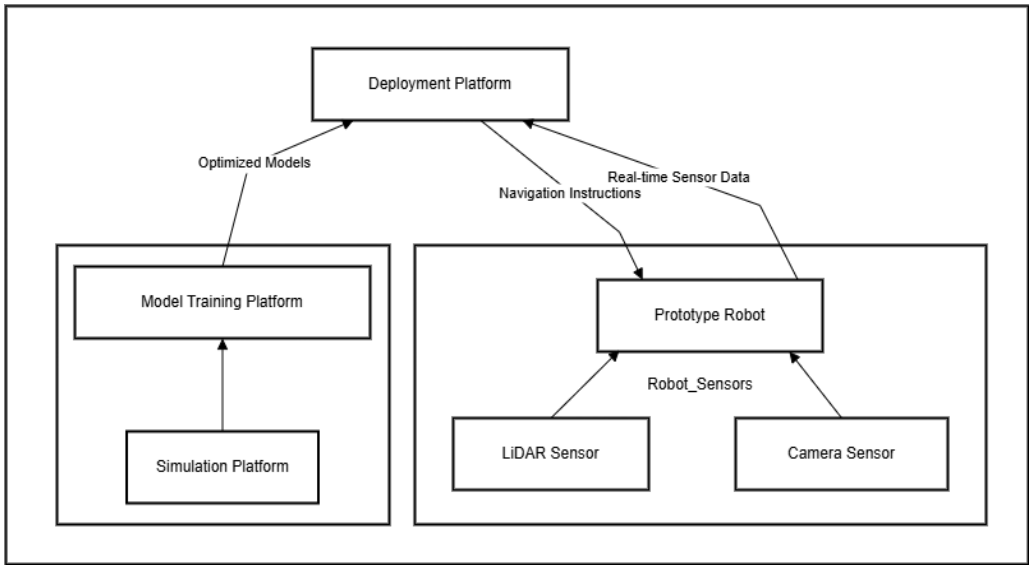

Figure 3.2 Sub Component Diagram

In order to accomplish the Multi-Robot Collaboration Framework project goals, the development process is divided into large work packages, each of which is further subdivided into distinct tasks to guarantee steady advancement and effective execution. These activities are organized to meet technical, operational, and validation criteria and are intended to be in line with the project timeframe. The system consists of three interconnected modules. The model training platform runs in a controlled simulation environment where LiDAR and camera inputs are emulated to mimic real-world conditions. The deployment platform acts as a bridge, transferring optimized models to the robot and enabling live monitoring. The prototype robot integrates the LiDAR and camera sensors, feeding real-time data into the onboard navigation stack. Data pipelines ensure that sensor data in simulation matches real-world formats, enabling smooth model transfer.

1. **System Architecture & Prototype Setup**
   - Finalize Hardware Selection: Choose robots, sensors and other hardware based on compatibility and cost.
   - Assemble and Configure Prototype Robots: First prototype comes together by combining a camera, a processing unit, and an Arduino-based control system

into a simple but functional loop. The camera acts as the primary sensor, continuously capturing visual input from the environment. These frames are processed on a laptop or Raspberry Pi, where the meta-reinforcement learning policy interprets the scene and decides the appropriate action, such as moving forward, turning, or stopping. The decision is then transmitted to the Arduino via wi-fi, which serves as the executor by converting high-level commands into low-level motor driver signals that control the wheels or tracks of the robot. To ensure safety during early tests, an LIDAR sensor can be connected directly to the Arduino, providing an emergency stop if obstacles are detected at close range. This modular setup vision and intelligence on the laptop or Pi, execution on the Arduino, and mobility through motors allows for a practical and scalable foundation, enabling quick iteration while leaving room for wireless communication and advanced sensors in later versions.

2. **Software Module Development**
   - Model training platform implementation.
   - Communication layer design.
   - Model deployment implementation.
3. **Integration & Deployment**
   - Integrate central server processing with cloud optimization.
   - Integrate simulation platform with training platform .
4. **Testing & Validation**
   - Unit and integration tests.
   - Scalability testing with multiple robots.
   - Real-world trials in controlled environments.
5. **Documentation & Handover**
   - System architecture documentation.
   - User manual creation.

3.4 Testing and Validation

Testing will be performed in progressive stages:

- **Unit Testing:** Each software module (map fusion, task allocation, conflict resolution) will be tested independently to verify functionality and accuracy.
- **Integration Testing:** Data flow between robots, central server, cloud, and visualization interface will be validated to ensure seamless interaction.
- **Performance Testing:** Metrics such as map fusion latency, task allocation fairness, and communication bandwidth usage will be measured to assess efficiency.
- **Scalability Testing:** Simulations with larger robot fleets under varied network and environmental conditions will evaluate system robustness.
- **Fault Tolerance Testing:** Network links will be disconnected to confirm autonomous fallback to local SLAM, ensuring safety during disruptions.

This staged approach ensures comprehensive validation across all components.

3.5 Data Collection

Data will be gathered from both simulated and real-world environments to support the development and validation of the Multi-Robot Collaboration Framework.

- **Simulated Data:** Generated using simulations to create diverse indoor scenarios for algorithm training, providing a controlled environment to test SLAM and Meta-RL models with varying conditions.
- **Real-World Data:** Collected from robot sensors in controlled indoor settings, such as university labs or mock warehouse spaces, to capture authentic environmental data and sensor noise.

The collected data will be processed into synchronized datasets, aligning sensor inputs with timestamps for use in SLAM map fusion and policy training, ensuring compatibility with the central server and cloud infrastructure.

3.6 Time Frame & Schedule

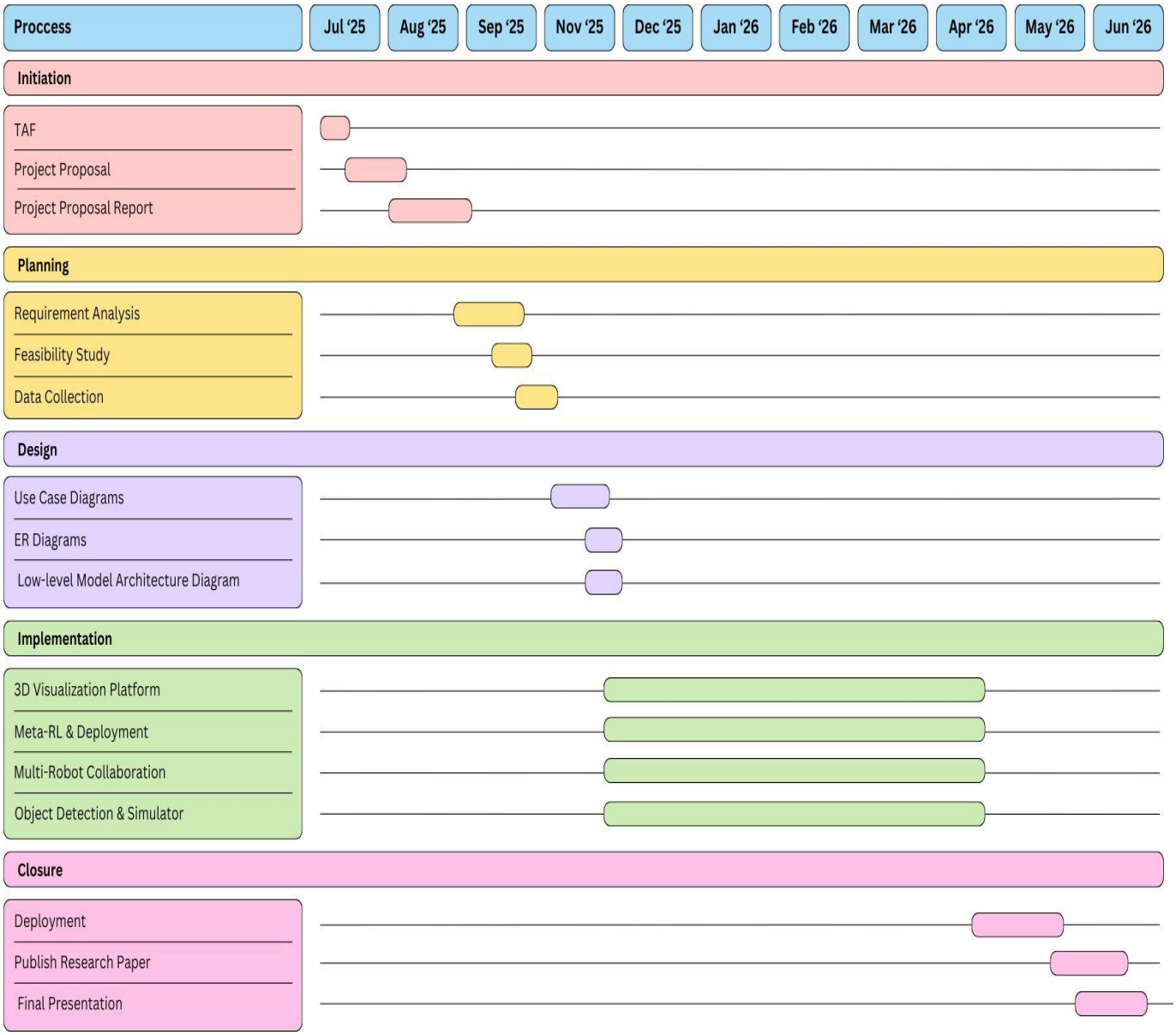

Figure 3.3: Gantt Chart

4 PROJECT REQUIREMENTS

4.1.1 Functional Requirements

The functional requirements define the specific features and capabilities that the system must deliver to achieve seamless model training, deployment, and real-world navigation. For the model training platform, the system must support importing LiDAR and camera data from both simulated and real environments, preprocessing it into synchronized datasets, and training deep learning or reinforcement learning models for path planning and obstacle avoidance. The platform should allow iterative training, where new data from field tests can be fed back into the training pipeline to improve accuracy.

For the deployment platform, the system must enable smooth transfer of trained models to the onboard computing unit of the robot, with minimal conversion or compatibility issues. It must support real-time inference from LiDAR and camera streams, integrating sensor fusion techniques to maintain consistent spatial awareness.

For the prototype robot, the system must integrate the LiDAR and camera sensors physically and logically, ensuring accurate calibration, time synchronization, and robust data transmission to the onboard processor. The robot must execute navigation commands in real time based on model outputs, and be capable of autonomous obstacle detection, mapping, and rerouting. Additionally, a real-time monitoring interface is required to visualize live sensor data, current navigation paths, and system diagnostics.

These functional requirements ensure that the system not only works within the lab environment but also transitions effectively to dynamic real-world conditions, supporting the iterative cycle of simulation, deployment, testing, and model refinement.

4.1.2 Non-Functional Requirements

Non-functional requirements define the quality attributes and performance constraints that the system must meet to ensure reliability, scalability, and usability. For the model training platform, performance and efficiency are critical. Training must be optimized to run within acceptable time frames, even with large LiDAR datasets, and the system must support parallel processing on GPUs or cloud-based compute clusters. The platform should also have modularity, allowing easy integration of new algorithms, sensors, or preprocessing techniques without major reengineering.

For the deployment platform, latency must be minimal to ensure that LiDAR and camera data are processed fast enough for real-time navigation. The system should be fault-tolerant, handling temporary sensor dropouts or degraded network conditions without significant performance loss. Security is another key aspect model files and deployment pipelines must be protected from unauthorized access or tampering.

For the prototype robot, robustness and durability are essential. The hardware must be capable of operating in various environments, from indoor spaces with tight corridors to outdoor areas with uneven terrain. Power efficiency is important to extend operational runtime, and thermal management must ensure stable processing under continuous load.

Usability is also a priority. The training and deployment platforms should have clear, intuitive interfaces for monitoring performance, adjusting parameters, and triggering retraining. Documentation and maintainability requirements ensure that future developers can understand, modify, and extend the system without steep learning curves. Collectively, these non-functional requirements guarantee that the system performs consistently, scales effectively, and remains adaptable for future research and real-world deployment scenarios.

4.1.3 User Requirements

Operators must use an easy-to-use interface to oversee and manage missions: Clear controls and graphics should be provided using the interface. Ability to override automatic decisions in an emergency: To improve safety and flexibility in critical situations, operators must have the ability to manually modify tasks or pathways.

4.1.4 System Requirements

Robots with LiDAR, RGB-D cameras, and IMUs: In order to gather unprocessed data and send it to the central server, edge devices need to be outfitted with these sensors. Central server with GPU acceleration: To effectively manage demanding map fusion and decision-making activities, the server needs GPU assistance. Cloud infrastructure for policy training: Meta-RL training and large-scale SLAM optimization with aggregated data require a cloud system.

REFERENCES

[1] Yu, C., Liu, Z., Liu, X.-J., Xie, F., Yang, Y., Wei, Q., and Fei, Q., "A Semantic Visual SLAM towards Dynamic Environments," arXiv preprint arXiv:1809.08379.

[2] Oriolo, G., Ulivi, G., and Vendittelli, M., "Real-time map building and navigation for autonomous robots in unknown environments," IEEE Transactions on Systems, Man, and Cybernetics, vol. 28, no. 3, pp. 316-333, 1998.

[3] Xing, Z., Zhu, X., and Dong, D., "DE-SLAM: SLAM for highly dynamic environment," Journal of Field Robotics, vol. 39, no. 4, pp. 528-549, 2022.

[4] Ai, Y., Rui, T., Yang, X., He, J., Fu, L., Li, J., and Lu, M., "Visual SLAM in dynamic environments based on object detection," Journal of Manufacturing Systems, vol. 60, pp. 257-271, 2021.

[5] Gan F, Xu S, Jiang L, Liu Y, Liu Q, Lan S. Robust visual SLAM algorithm based on target detection and clustering in dynamic scenarios. Front Neurorobot. 2024 Jul 23;18:1431897. doi: 10.3389/fnbot.2024.1431897. PMID: 39108349; PMCID: PMC11300432.

[6] Li, F., Yang, S., Yi, X., and Yang, X., "CORB-SLAM: A Collaborative Visual SLAM System for Multiple Robots," in Proc. International Conference on Intelligent Robotics and Applications, 2018, pp. 377-388.

[7] Ahmed, M. F. et al., "Efficient Frontier Management for Collaborative Active SLAM," arXiv preprint arXiv:2310.01967, 2023.

[8] Ducatelle, Frederick & Förster, Alexander & Bonani, Michael & Dorigo, Marco & Magnenat, Stéphane & Mondada, Francesco & O'Grady, Rehan & Pinciroli, Carlo & Retornaz, Philippe & Trianni, Vito & Gambardella, Luca Maria. (2014). Cooperative navigation in robotic swarms. Swarm Intelligence. 10.1007/s11721-013-0089-4.

[9] Wu X, Wang G and Shen N (2023) Research on obstacle avoidance optimization and path planning of autonomous vehicles based on attention mechanism combined

with multimodal information decision-making thoughts of robots. Front. Neurorobot. 17:1269447. doi: 10.3389/fnbot.2023.1269447

[10] Raj, R. and Kos, A., "Intelligent mobile robot navigation in unknown and complex environment using reinforcement learning technique," Scientific Reports, vol. 14, no. 1, p. 22852, 2024.

[11] Fu, Z., Kumar, A., Malik, J., and Pathak, D., "Rapidly Adaptable Legged Robots via Evolutionary Meta-Learning," Google Research, 2020.

[12] Blanco, Jose Luis. (2019). A Modular Optimization Framework for Localization and Mapping. 10.15607/RSS.2019.XV.043.

[13] R. Mur-Artal, J. M. M. Montiel, and J. D. Tardós, “ORB-SLAM: A versatile and accurate monocular SLAM system,” IEEE Trans. Robot., vol. 31, no. 5, pp. 1147–1163, Oct. 2015.

[14] N. Sünderhauf, et al., “Place Recognition with ConvNet Features: Static vs Dynamic Environments,” IEEE Robot. Autom. Lett., vol. 3, no. 3, pp. 2225–2232, Jul. 2018.

[15] D. Kim, J. Park, and S. Lee, “Semantic-Aware Change Detection for Long-Term SLAM,” Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS), pp. 320–327, 2019.